

Long Short Term Memories and Gated Recurrent Units

Jonathon Hare

Some of the images and animations used here were originally designed by Adam Prügel-Bennett.

Recap: An RNN is just a recursive function invocation

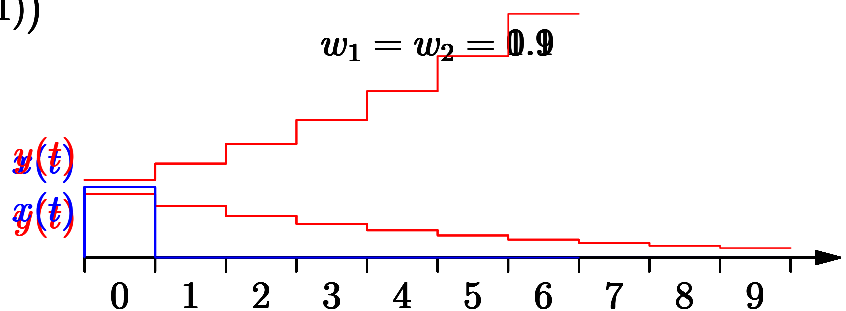
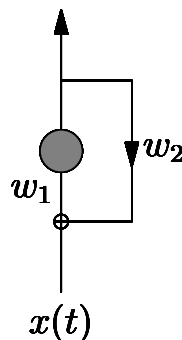
- $y(t) = f(x(t), c(t) | W_f)$
- and the state $c(t) = g(x(t), c(t-1) | W_g)$
- If the output $y(t)$ depends on the input $x(t-2)$, then prediction will be

$$f(x(t), g(x(t), g(x(t-1), g(x(t-2), c(t-2) | W_g) | W_g) | W_g) | W_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule
- The back-propagated error will involve applying f multiple times
- Each time the error will get multiplied by some factor a
- If $y(t)$ depends on the input $x(t-\tau)$ then the back-propagated signal will be proportional to $a^{\tau-1}$
- This either vanishes or explodes when τ becomes large

Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$



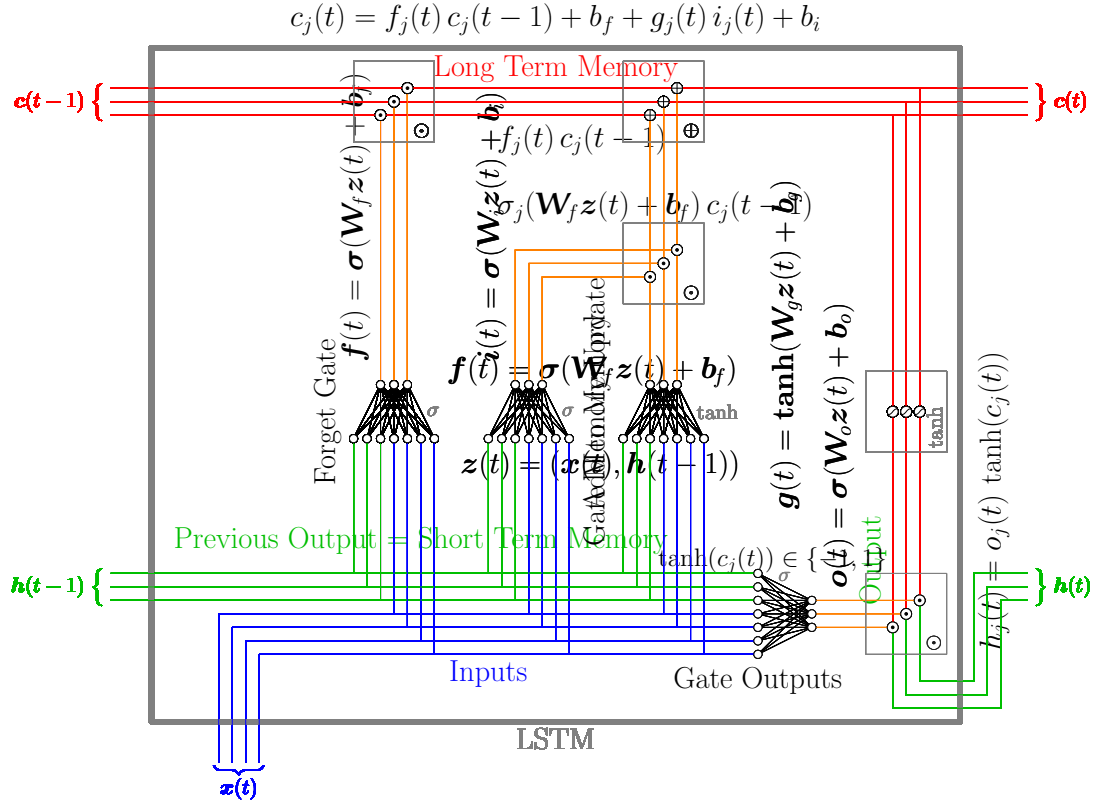
LSTM Architecture

- The LSTM (long-short term memory) was designed to solve this problem
- Key ideas: to retain a 'long-term memory' requires

$$c(t) = c(t-1)$$

- Sometimes we have to forget and sometimes we have to change a memory
- To do this we should use 'gates' that saturate at 0 and 1
- Sigmoid functions naturally saturate at 0 and 1

LSTM Architecture



Update Equations

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

- Inputs $\mathbf{z}(t) = (\mathbf{x}(t), \mathbf{h}(t-1))$
- Network updates ($\mathbf{W}_*$ and $\mathbf{b}_*$ are the learnable parameters)

$$\begin{aligned} \mathbf{f}(t) &= \sigma(\mathbf{W}_f \mathbf{z}(t) + \mathbf{b}_f) & \mathbf{i}(t) &= \sigma(\mathbf{W}_i \mathbf{z}(t) + \mathbf{b}_i) \\ \mathbf{g}(t) &= \tanh(\mathbf{W}_g \mathbf{z}(t) + \mathbf{b}_g) & \mathbf{o}(t) &= \sigma(\mathbf{W}_o \mathbf{z}(t) + \mathbf{b}_o) \end{aligned}$$

- Long-term memory update

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{g}(t) \odot \mathbf{i}(t)$$

- Output $\mathbf{h}(t) = \mathbf{o}(t) \odot \tanh(\mathbf{c}(t))$

Training LSTMs

- We can train an LSTM by unwrapping it in time.
- Note that it involves four dense layers with sigmoidal (or tanh) outputs.
- This means that typically it is very slow to train.
- There are a few variants of LSTMs, but all are very similar. The most popular is probably the Gated Recurrent Unit (GRU).

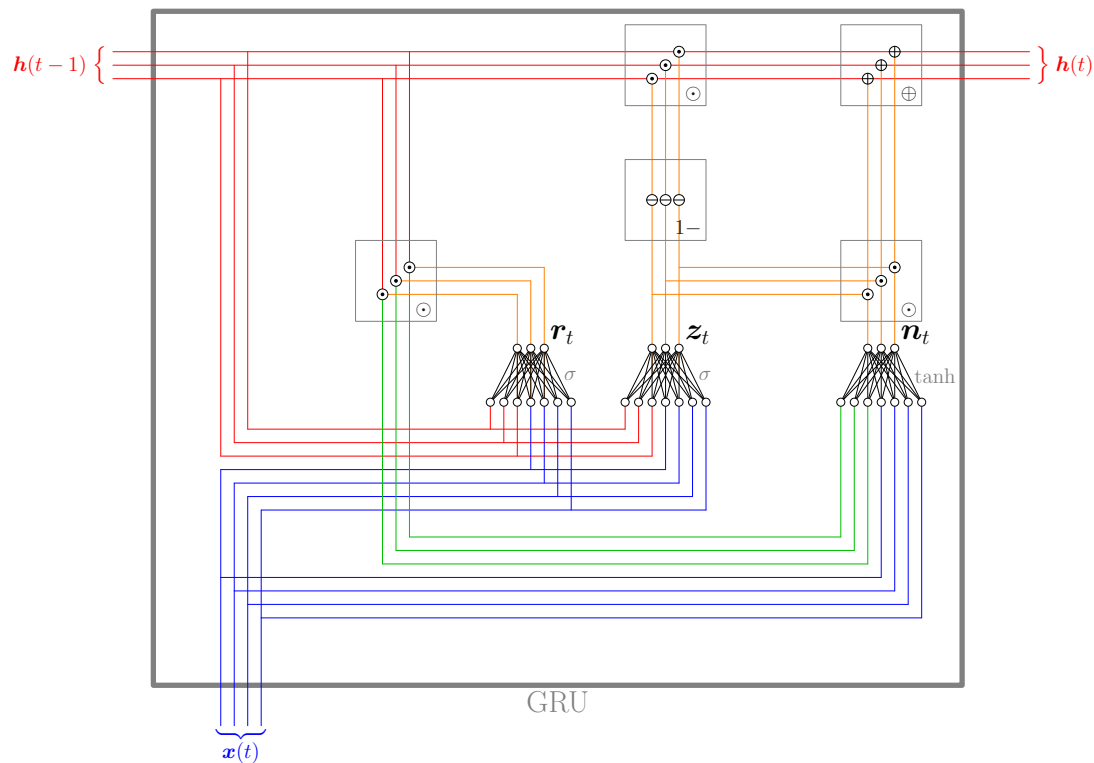
pause

LSTM Success Stories

- LSTMs have been used to win many competitions in speech and handwriting recognition.
- Major technology companies including Google, Apple, and Microsoft are using LSTMs as fundamental components in products.
- Google used LSTM for speech recognition on the smartphone, for Google Translate.
- Apple uses LSTM for the "Quicktype" function on the iPhone and for Siri.
- Amazon uses LSTM for Amazon Alexa.
- In 2017, Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks¹.

pause

Gated Recurrent Unit (GRU)



pause

Gated Recurrent Unit (GRU)

- $x(t)$: input vector
- $h(t)$: output vector (and 'hidden state')
- $r(t)$: reset gate vector
- $z(t)$: update gate vector
- $n(t)$: new state vector (before update is applied)
- W and b : parameter matrices and biases

¹https://en.wikipedia.org/wiki/Long_short-term_memory

pause

Gated Recurrent Unit (GRU)

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

$$\begin{aligned}\mathbf{z}(t) &= \sigma(\mathbf{W}_z(\mathbf{x}(t), \mathbf{h}(t-1)) + \mathbf{b}_z) \\ \mathbf{r}(t) &= \sigma(\mathbf{W}_r(\mathbf{x}(t), \mathbf{h}(t-1)) + \mathbf{b}_r) \\ \mathbf{n}(t) &= \tanh(\mathbf{W}_n(\mathbf{x}(t), \mathbf{r}(t) \odot \mathbf{h}(t-1)) + \mathbf{b}_n) \\ \mathbf{h}(t) &= (1 - \mathbf{z}(t)) \odot \mathbf{h}(t-1) + \mathbf{z}(t) \odot \mathbf{n}(t)\end{aligned}$$

pause

GRU or LSTM?

- GRUs have two gates (reset and update) whereas LSTM has three gates (input/output/forget)
- GRU performance on par with LSTM but computationally more efficient (less operations & weights).
- In general, if you have a very large dataset then LSTMs will likely perform slightly better.
- GRUs are a good choice for smaller datasets.

Most implementations follow the original paper and swap $(1 - \mathbf{z}(t))$ and $(\mathbf{z}(t))$ in the $\mathbf{h}(t)$ update; this doesn't change the operation of the network, but does change the interpretation of the update gate, as the gate would have to produce a 0 when an update was to occur, and a 1 when no update is to happen (which is somewhat counter-intuitive)!